

	ORIGINATE DATE 11 March, 2001	EDIT DATE 4 September, 2015	DOCUMENT-REV. NUM. GEN-CXXXXX-REVA	PAGE 1 of 32
Author: Steve Morein				
Issue To:			Copy No:	
R400 Top Level Specification ver 0.2				
Overview: This replaces the R400 architecture specification.				
AUTOMATICALLY UPDATED FIELDS: Document Location: Document1 Current Intranet Search Title : R400 Top Level Spec				
APPROVALS				
Name/Dept		Signature/Date		
Remarks:				
THIS DOCUMENT CONTAINS CONFIDENTIAL INFORMATION THAT COULD BE SUBSTANTIALLY DETRIMENTAL TO THE INTEREST OF ATI TECHNOLOGIES INC. THROUGH UNAUTHORIZED USE OR DISCLOSURE.				
“Copyright 2000, ATI Technologies Inc. All rights reserved. The material in this document constitutes an unpublished work created in 2000. The use of this copyright notice is intended to provide notice that ATI owns a copyright in this unpublished work. The copyright notice is not an admission that publication has occurred. This work contains confidential, proprietary information and trade secrets of ATI. No part of this document may be used, reproduced, or transmitted in any form or by any means without the prior written permission of ATI Technologies Inc.”				

	ORIGINATE DATE 11 March, 2001	EDIT DATE 4 September, 2015	DOCUMENT-REV. NUM. R400 Top Level Spec	PAGE 2 of 32
--	----------------------------------	--------------------------------	---	-----------------

Table Of Contents

1. FEATURES	7
1.1 AGP 8x	7
1.2 256 Bit Memory Interface	7
1.3 Unified Processing pipe	7
1.4 Front end scaling	7
1.5 Real-Time drawing command ability	7
1.6 3D Features	8
1.6.1 Noise Textures	8
1.6.2 Shadow buffers	8
1.6.3 Sort Independent Transparency	8
1.6.4 Anti-Aliasing	8
1.6.5 Texture compression	8
1.6.6 Z compression	8
1.6.7 Texture Filtering	8
1.6.8 Curved Surface Support	9
1.6.9 Displacement maps	9
1.7 High color depth	9
2. PERFORMANCE	9
3. SCHEDULE	9
4. PROCESS	9
5. GENERAL CHIP OPERATION	9
5.1 Unified Shader	9
5.2 3D Rendering	10
5.3 Real Time Rendering	12
5.4 State Management	13
5.5 Bad Data	13
5.6 Display operation	14
6. BLOCK DIAGRAM	15
7. BLOCKS	15
8. BLOCK DESRIPTONS	16
8.1 HBIU – host bus interface unit	16
8.1.1 Description	16
8.1.2 Major interfaces	16
8.1.3 Block diagram	17
8.2 CP – control processor	18
8.2.1 Description	18
8.2.2 Major interfaces	18
8.2.3 Block diagram	19
8.3 RBBM – register interface manager	19
8.3.1 Description	19

PROTECTIVE ORDER MATERIAL

	ORIGINATE DATE 11 March, 2001	EDIT DATE 4 September, 2015	DOCUMENT-REV. NUM. GEN-CXXXXXX-REVA	PAGE 3 of 32
--	----------------------------------	--------------------------------	--	-----------------

8.3.2	Major interfaces.....	19
8.3.3	Block diagram.....	20
8.3.4	RBBM operation	20
8.4	CLK – clock generator.....	21
8.4.1	Description	21
8.4.2	Major interfaces.....	21
8.4.3	Block diagram.....	22
8.5	TC – test controller	22
8.5.1	Description	22
8.5.2	Major interfaces.....	22
8.5.3	Block diagram.....	22
8.6	VIP – Video input port	22
8.6.1	Description	22
8.6.2	Major interfaces.....	22
8.6.3	Block diagram.....	22
8.7	ROM – boot rom.....	22
8.7.1	Description	22
8.7.2	Major interfaces.....	22
8.7.3	Block diagram.....	22
8.8	I2C – I2C interface	22
8.8.1	Description	22
8.8.2	Major interfaces.....	22
8.8.3	Block diagram.....	23
8.9	DU – Display	23
8.9.1	Description	23
8.9.2	Major interfaces.....	23
8.9.3	Block diagram.....	24
8.10	MH – Memory Hub	24
8.10.1	Description	24
8.10.2	Major interfaces.....	24
8.10.3	Block diagram.....	24
8.11	HDP – Host Data Path	24
8.11.1	Description	25
8.11.2	Major interfaces.....	25
8.11.3	Block diagram.....	25
8.12	IDCT – Mpeg decoder	25
8.12.1	Description	25
8.12.2	Major interfaces.....	25
8.12.3	Block diagram.....	25
8.13	PA – Primitive Assembly	25

PROTECTIVE ORDER MATERIAL

	ORIGINATE DATE 11 March, 2001	EDIT DATE 4 September, 2015	DOCUMENT-REV. NUM. R400 Top Level Spec	PAGE 4 of 32
--	---	---------------------------------------	--	------------------------

8.13.1	Description	25
8.13.2	Major interfaces	25
8.13.3	Block diagram.....	26
8.14	TD – Texture Decompression	26
8.14.1	Description	26
8.14.2	Major interfaces	27
8.14.3	Block diagram.....	27
8.15	RE – Raster Engine.....	27
8.15.1	Description	27
8.15.2	Major interfaces	28
8.15.3	Block diagram.....	29
8.16	SP – Shader Pipe	30
8.16.1	Description	30
8.16.2	Major interfaces	30
8.16.3	Block diagram.....	31
8.17	TP – Texture Pipe	32
8.17.1	Description	32
8.17.2	Major interfaces	32
8.17.3	Block diagram.....	32
8.18	RB – Render Backend.....	32
8.18.1	Description	32
8.18.2	Major interfaces	32
8.18.3	Block diagram.....	32
8.19	MC – Memory Controller	32
8.19.1	Description	32
8.19.2	Major interfaces	32
8.19.3	Block diagram.....	32
9.	COMMON FOUNDATIONS	32
9.1	Logic Design	32
9.1.1	Data formats	32
9.1.2	Register Bus	32
9.1.3	Block Communication protocol	32
9.2	Software	32

PROTECTIVE ORDER MATERIAL

	ORIGINATE DATE 11 March, 2001	EDIT DATE 4 September, 2015	DOCUMENT-REV. NUM. GEN-CXXXXXX-REVA	PAGE 5 of 32
--	---	---	---	----------------------------

Revision Changes:

Rev 0.0 (Steve Morein)

Date: March 11, 2001

Initial revision.

Document recreated from earlier documents

Date March 14,2001

Finally got back to editing it.

PROTECTIVE ORDER MATERIAL

	ORIGINATE DATE 11 March, 2001	EDIT DATE 4 September, 2015	DOCUMENT-REV. NUM. R400 Top Level Spec	PAGE 6 of 32
--	----------------------------------	--------------------------------	---	-----------------

Introduction

The R400 will be the high end standalone graphics chip product when it is introduced.

It will be followed very rapidly with two variants:

The RV400, aimed at the volume PC space

The R450, aimed at a volume high end market.

The targets for the three chips are:

Part	Clock Speed	pixels/clock	texture fetches/clock	alu ops/clock	Memory width	Memory speed	die size	Tapeout
R400	400 MHz	8	16	32	256	400MHz	11.5	July,2002
RV400	500 MHz	4	8	16	128	500 MHz	8.5	Nov 2002
R450	500 MHz	8	16	32	256?	500 MHz	9.5	Feb 2003

	ORIGINATE DATE 11 March, 2001	EDIT DATE 4 September, 2015	DOCUMENT-REV. NUM. GEN-CXXXXX-REVA	PAGE 7 of 32
--	----------------------------------	--------------------------------	---------------------------------------	-----------------

1. Features

1.1 AGP 8x

The chip will support the 32 bit AGP interface at speeds up to 8x. I expect that we will need to support AGP 1x and 2x which require 3.3 Volt I/O (AGP 4x is 1.5v and AGP 8x is 750mv). AGP fast writes are supported for access to the frame buffer.

Open issue: 64 bit address space support.

1.2 256 Bit Memory Interface

The R400 and R450 support four memory channels, which can be 32 or 64 bits wide; the maximum memory bus width is a total of 256 bits. The RV400 supports two memory channels and a maximum total width of 128 bits.

All channels need to be configured identically, 1, 2 or 4 channels can be configured.

Memory standards supported:

I/O	Voltage	Memory type	Speed
SSTL2.5	2.5	DDR	100 to 500 MHz
SSTL1.8	1.8	DDR/infineon	300 to 500 MHz
Elpida	1.8 (1.5?)	Elpida	300 to 400 MHz
Infineon	1.2, 1.0 V	Infion e-dram	500 MHz

No support for SSTL3.3, or SDRAM (LVTTTL – 3.3V) is planned.

1.3 Unified Processing pipe

The most ambitious feature in this design is the “truly unified pipe” : a single programmable pipeline is used for 2D, Video, 3D vertex, and 3D pixel operations. The unified pipeline does all of its calculations in 32 bit floating point, the same as the existing vertex transform in previous chip, and the next step in the precision of the color/pixel calculations which have increased from 8 bits (R100), through 16 bits (R200), to the 20 bits in the R300.

There is an area cost to the unified pipeline since we are forced to go to 32 bit precision for color, when application requirements may need less (22 to 24 bits). However the unified pipeline results in a single math/register structure compared to the separate structures in a more traditional design. It is hoped that by only needing to design the one structure we can make the investment in design time and effort to really optimize the area.

Some of the benefits to merging the pipelines include allowing the vertex operations to do texture fetches, which we could not afford add logic to the transform pipe to do, a single programming model for both operations, more precision on color than we would normally provide, and the ability to support significantly more registers and instructions in pixel shaders.

One important benefit is load balancing. In the current pipeline when the app it transform bound the pixel pipeline is idle some significant portion of the time, and when the app is raster bound the transform hardware idle. The unified pipeline presented here dynamically allocates its processing power between transform and raster.

1.4 Front end scaling

We will remove the back end scaling capability from the display, and replace it with a non-scaling overlay. This will require us to be able to implement scaling using the unified pipeline. Key features that will need to be supported are large filter kernels, de-interlacing, frame rate conversion, and good support for YUV and color conversion.

1.5 Real-Time drawing command ability

To allow for the emulation of backend scaling as well as support new features we need to be able to interrupt the 3D pipe and be able to execute high priority commands with low latency. The point of interruption is in the primitive

	ORIGINATE DATE 11 March, 2001	EDIT DATE 4 September, 2015	DOCUMENT-REV. NUM. R400 Top Level Spec	PAGE 8 of 32
--	----------------------------------	--------------------------------	---	-----------------

assembly, the maximum latency will be about the time it takes to render 4096 pixels. The real time commands are inserted into the 3D pipeline after transform, clipping, and setup. Those function need to be performed by the driver. There are also limits on the number of constant registers available.

1.6 3D Features

There are a number of new 3D features we are considering for inclusion. Additional features may be added, and some of these may be dropped.

1.6.1 Noise Textures

Perlin style noise is useful for a number of applications. It is generated on chip and consumes no external memory bandwidth. It also larger than any physical texture can be: 256x256x256 lattice points, and still has detail when the resolution is 4Kx4Kx4K. There is an opportunity to get this adopted as part of dx9.

1.6.2 Shadow buffers

John Carmack is using shadow volumes to generate shadow effects in doom3. Shadow volumes are very poor way to use modern 3D pipelines. (will add more detail here later). Shadow buffers have two key limitations: very high resolutions are required to avoid aliasing, and traditional shadow buffers can not be mip-mapped so filtering is real problem. We are able to solve the first problem through a combination of our improved anti-aliasing Z compression, and a new method of implementing the shadow map probe.

1.6.3 Sort Independent Transparency

We are currently looking into how best to support sort independent transparency. The two plans are either the dual Z buffer approach, or the approach described in <need to decide where the email should be placed so others can see>

1.6.4 Anti-Aliasing

The changes from the R300 include an increased number of samples per pixel, probably eight, and support for an allocated frame buffer size smaller than the worst case maximum.

1.6.5 Texture compression

To further reduce bandwidth we need to improve texture compression. We need to achieve both better compression than S3TC, and have a high enough quality that textures that would lose too much detail with S3TC can be compressed. Both of these goals do not need to be achieved simultaneously on all textures. We also need to look at compression of non-traditional surfaces such as normal maps. Advances here are dependent on the availability of resources to work on this. If we are unable to find resources we will support the s3tc compression currently in D3D.

1.6.6 Z compression

<larry needs to give me a paragraph here>

1.6.7 Texture Filtering

The texture pipes can fetch a 2x2 region from the texture map and filter it.

The data per pixel can either be four eight bit values, two sixteen bit values, or one 32 value. All data needs to be fixed point.

Linear filters are completely built in, and it takes 1 cycle for bi-linear, 2 for tri-linear, four for quadra-linear (filtered mip-mapping of volume textures). Variable depth anisotropy is supported in hardware with the texture pipe calculating the number of samples needed. Optionally the pixel shader can calculate the number of samples, and how to increment the texture address, and provide this to the texture pipe.

	ORIGINATE DATE 11 March, 2001	EDIT DATE 4 September, 2015	DOCUMENT-REV. NUM. GEN-CXXXXX-REVA	PAGE 9 of 32
--	----------------------------------	--------------------------------	---------------------------------------	-----------------

1.6.8 Curved Surface Support

We will support curved surfaces through combination of vertex shader code and a tessellation engine to generate new vertices.

The tessellation engine generated new vertex indices from a input vertex index array. The new indices contain both the coordinate in parametric space of the vertex, and the indices to the surface, or to data from which the surface can be derived. More information is available in the programming guide.

1.6.9 Displacement maps

The tessellation engine for curved surfaces can dice triangles into micropolygons, the vertex shaders for the vertices can then access into a displacement map and change the location of the points.

1.7 High color depth

We will support a 64 bit color buffer (16:16:16:16), we will support two formats: sRGB64 and a floating point format..
<need to insert format details.

2. Performance

The basic performance is:

R400 MHz fill rate bi-linear equiv peak tri/sec

	MHz	Fill rate	Bi-linear texture fetches	Peak tri/sec
R400	400	3.2 gigapixel	6.4 Billion	400 Million
RV400	500	2.0	4.0	500 Million
R450	500	4.0	8.0	500 Million

Under normal conditions, and when not further limited by memory bandwidth we expect to be > 75% efficient.

3. Schedule

		Tapeout	Samples	Production
R400		July, 2002	Oct, 2002	Dec, 2002
RV400		Nov, 2002	Jan, 2003	March, 2003
R450		Jan, 2003	April 2003	May 2003

4. Process

At the moment this looks like an easy choice: .13 will be in production for over a year, and .10 does not show up until the very end of 2002 according to the TSMC and UMC roadmaps.

We will probably want to be in a flip chip packaging approach to meet power distribution goals. With the 256 bit bus we will have at least 600 signal I/O's (404 in memory). We may be as much as 10A at 1V for average power, which will require very good power distribution, area bond flip chip is probably the only option.

5. General Chip operation

5.1 Unified Shader

The unified shader is a simd/vector engine that performs the same instructions on four sets of four (16 total) elements. For pixel shader operations the elements are pixels with the sets of four required to be 2x2 footprints. For

	ORIGINATE DATE 11 March, 2001	EDIT DATE 4 September, 2015	DOCUMENT-REV. NUM. R400 Top Level Spec	PAGE 10 of 32
--	----------------------------------	--------------------------------	---	------------------

vertex shader operations the sixteen elements are sixteen vertices. The basic element is a 4 value vector – frequently interpreted as x,y,z,w or r,g,b,a.

The user model for the unified shader is composed of a variable number of general purpose registers, a subset of which are usually initialized with data. An ALU can do simple math, conditional moves, and permutations on the registers, and the ability to do a limited number of memory reads using the texture cache. The number of register is variable, and the number of registers required for an operation are specified when the task is submitted to the unified shader. The unified shader will not start the task until there is enough free room for the task's registers.

The unified shader is based on the R300 pixel shader.

5.2 3D Rendering

For 3D rendering data is passed twice through the unified shader- once to transform the vertices and a second time to determine the color of the pixels.

The input to the 3D pipe is expected to be indexed vertex arrays. Linear vertex arrays can easily be supported by the CP generating sequential indices. Inline vertex data is an open issue, I would prefer to write it to memory and then fetch it as a vertex array rather than add a direct path.

The stream of indices is sent to the Primitive Assembly block by the CP. The front of the primitive assembly block maintains the tag for the vertex cache; The vertex cache stores transformed vertices. As misses are detected in the tag, the indices that miss are placed into 16 entry vectors. Each vector contains a state pointer, a pointer to the vertex shader to be used, and the 16 indices to vertices that need to be transformed. When either a vector is filled with 16 entries or a state change happens (so that the next vertex does not share the state and vertex shader with the previous vertex) the vector is issued to one of the "shader" pipelines for transformation. Which of the four shader pipelines it is issued to determined either by some effort of load balancing or a simple round robin. All that is submitted to the pixel pipeline is the state, the vertex program, and the indices. The shader pipeline will fetch the vertex array data through the cache infrastructure that is also used for texture fetches. After the tag the indices (actually now the indices into the vertex cache) are placed into a latency FIFO to hide the latency of transforming the vertices.

The shader pipeline receives the vector of 16 indices from the primitive assembly block. The shader pipeline operates, when rendering pixels, by processing a vector of four 2x2 pixel footprints, a total of 16 pixels. For vertex processing each of the pixels is replaced with a vertex. The vertex program includes information of how many local variables it will need. The rasterizer waits until that many local variables are free, (as each executing thread in the shader pipeline terminates it frees its local variables). With the proposed shader data path the maximum number of local variables per vertex is 256. However this leaves no ability to hide latency, 16 to 32 local variables will probably maximize latency hiding and therefore performance. The vertex shader program can use all the capabilities of the shader pipeline including texture fetches and dependent lookups. At the end of the vertex program, the transformed coordinates must be output. One output will be the x, y, z, w position which we be stored in the position cache of the vertex cache. The vertex program may also output a number of parameter values (colors, texture coordinates, other interpolated inputs into the pixel shader). The parameter values must be output as a multiple of four 128 bit words, as the parameter cache is designed for this.

The primitive assembly block reads the indices back out of the latency FIFO and accesses the position cache portion of the vertex cache. It assembles the vertices into primitives (lines, triangles, rectangles, quads?, points, ?). Baricentric values are assigned to the vertices, and will be used later in the rasterizer to interpolate the parameters. The parameters are not accessed by the primitive assembly logic, which only works from the position data. The primitive is clipped against both the viewing volume as well as user clip planes, with fractional baricentric coordinates assigned to the clipped primitive sections. The primitive goes through the perspective divide and the viewport transform. The resulting screen space primitive is setup (plane equations for 1/W, Z, and the baricentric coordinates). The resulting primitive data, including the indices back into the parameter portion of the vertex cache are broadcast to the four pipes. The final time that an index is output that access the oldest vertex cache line, a token is also sent. When all of the four pipelines return the token the primitive assembly block can free that cacheline and allow it to be used for a new vector of vertices. The performance goal in the primitive assembly block is a triangle every two clocks. An alternative option is for the vertex shader to generate screen coordinates and clip codes. If a primitive needs to be clipped, which can not be determined until primitive assembly, then the vertices are reverse transformed back into clip space by logic in the primitive assembly block, clipped, and then transformed back into screen space.

PROTECTIVE ORDER MATERIAL

	ORIGINATE DATE 11 March, 2001	EDIT DATE 4 September, 2015	DOCUMENT-REV. NUM. GEN-CXXXXX-REVA	PAGE 11 of 22
--	---	---	--	-----------------------------

To help meet marketing BS numbers we can look into doing backface culling at a rate of one triangle per clock. This will boost us to peak bs number of 500 million triangles per second.

Each pipe has a FIFO in front of the rasterizer to load balance. Each pipe will handle 16x16 tiles of the screen which are interleaved between the pipes. To maximize the effective size of the FIFO we will probably cull the triangle list before the FIFO. The rasterizer will request the parameter data from the parameter cache for the primitives. A small latency hiding FIFO will hide the latency of the access to the parameter cache. The parameter cache is 512 bits wide, and the interfaces from the parameter cache to the rasterizer are 128 bits wide, this allows the parameter cache to output one pipelines request per clock, which is serialized over four clocks, keeping all four interfaces busy. The rasterizer keeps a small cache of three to four vertices, this allow only the new parameter to be fetched when adjacent triangles are processed. The parameter cache interface imposes a second performance limits, in the worst case each polygon covers all four pipelines and there are no vertices shared from triangle to triangle. In this case the peak performance is $(500 \text{ MHz} / (4 \text{ pipelines} * 3 \text{ vertices})) = (500/12) = 41.6 \text{ million triangles per second}$. In the best case triangles are perfectly stripped and never cross over pipeline boundaries. In this case the peak performance (If we ignore the setup limit) is 500 million triangles per second. As a practical manner we should be able to approach the setup limit of 250 Million triangles per second.

The rasterizer also contains a portion of the hierarchical Z memory. We are looking into moving this into a cache based approach, but that is far for certain at this point. We would like to be able to do hierarchical z culling at a speed in excess of 64 pixel per clock per pipelines (256 pixels per clock total). We are also going to consider some of the improved latency hierarchical Z options to improve culling efficiency.

The rasterizer will generate four pixels per clock if there are no more than eight interpolated parameters. The rasterizer generates vectors of four 2x2 footprints (16 pixels). Each 2x2 footprint must be screen aligned and from the same triangle (with a single shared z slope). The four footprints only need to share the same state and shader program.

Before starting the processing of a vector the rasterizer (which includes the sequencer for the shader pipeline) checks to make sure that there are enough free registers in the shader pipeline for the pixel shader program. If not, it stalls until there are enough. The rasterizer also needs to arbitrate between the three streams of vectors to be shaded: the vertex stream, the pixel stream, and the real time stream. I think it will be sufficient for the real time stream to have priority over the vertex stream which has priority over the pixel stream. This will meet the real-time demands, and keep the vertex cache filled.

The vector is then processed by the shader pipeline. We will probably support up to eight sequentially dependent texture fetches. (to use the R300 terminology, eight clauses). 16 (8?) textures are supported, but each texture can be accessed multiple times by a single pixel shader which can provide a different address each time. This is especially useful for complex filters.

The output of the pixel shader is the final color of the fragment. The pixel shader may also replace the Z value. Fog and stippling must be done in the pixel shader program.

The render backend does the z compare, stencil operation and color alpha blend.

The texture fetch path has a number of design options. One option is an approach where the local, multiported, texture cache is small (1 to 4 KB), and contains uncompressed color in a canonical format (32 bits per pixel) and uses a 4x2 or 4x4 cacheline. This is backed up by a large (>16KB) L2 cache which also stored uncompressed 8x8 cachelines. The decompression logic lives between the memory controller and the L2 cache.

An alternative design uses the L2 cache to contain data in memory format (compressed) which is decompressed as needed to fulfill L1 texture cache misses. This will increase the effective size of the L2. The L2 cache is distributed, with 1/4 of it residing in each memory controller. The Texture decompression logic can either be located in each shader pipeline, or exist as a shared block(s) that receive data from all four memory controller and send the decompressed 4x4 cachelines to each shader pipeline. The unified decompression block will result in better performance, and possibly less area, at the cost of some of the scalability.

Assuming that we chose the L2 in memory controller and the unified decompression logic, the texture path would work as follows:

	ORIGINATE DATE 11 March, 2001	EDIT DATE 4 September, 2015	DOCUMENT-REV. NUM. R400 Top Level Spec	PAGE 12 of 32
--	----------------------------------	--------------------------------	---	------------------

In a four pipeline design there are two texture decompression blocks, one for the "left" texture units in each shader pipeline, and the second for the "right" texture units. In the two pipeline, lower cost, version of the chip only a single decompression pipeline is used, serving the left and right texture units.

The L1 texture cache receives a texture request from its shader pipeline. The usual tag and latency FIFO is used to generate the misses. These are sent to the shared texture decompression block, which looks up the texture to find the physical address and then sends the request to the L2 cache in the memory controller. The L2 also has a latency FIFO and tag, and will return the data in order (but there is no order guaranteed between the data returning from each L2). The decompression block has a buffer which is used to place the data from the memory controllers back in order. The decompression logic decompresses the texture and returns, in order, the 4x4 cachelines that the L1 caches are requesting. Most of the compression techniques we are considering are based on an 8x8 tile (or 4x4x4), when necessary the decompression logic will decompress an entire 64 pixel tile and only return the requested 16 pixels to the L1 cache. This will tend to increase the bandwidth between the decompression logic and the L2 cache as 8x8 blocks are repeatedly requested to provide different 4x4 subtiles to the L1. The L2 cache will prevent the repeated reads from going to memory, and we will probably implement an "L0" style cache in front of the L2 to also catch the redundant requests.

Each memory controller will have two 64 bit read return buses, one to each of the two decompression blocks, each decompression blocks drives a separate 128 bit bus to each of the four shader pipelines. This will tend to have better utilization and load balancing than having the memory controller drive a 32 bit bus to the decompression logic in each shader pipeline. While the total number of wires is similar (128 bits per memory controller, 128 bits into each texture cache) we are less likely to leave the texture pipes starved when there is some imbalance.

5.3 Real Time Rendering

The real time rendering interface allows primitives to be inserted into the rendering pipeline at a very late stage, therefore providing very low latency. The expected use is for scale blits timed by the display refresh, this suggests a small number of large primitives. We take advantage of this to simplify hardware by forcing the interface to be post setup, a real-time primitive needs to be transformed and setup by software.

Real time primitives also do not have access to the state management hardware used by non-real-time 3D commands. A single set of state registers, some constant registers, and one full parameter set is available. The real-time command stream will generally need to wait for the current real-time drawing operation to complete before it can start the next real-time command. The driver can statically allocate some of the physical constant registers to the real-time stream, these are not available to the RBBM for renaming use, and are written by the real-time command stream, and read by the 3D pipe at the direct physical addresses. There are two options for the parameter memory. The parameter memory is not visible to non-real-time commands, for normal operation it is entirely managed by hardware. For real time rendering there will be dedicated space for three vertices, each with sixteen 128 bit interpolants. If the real-time primitive requires more than eight interpolants there will only be enough room for one primitive at a time, even if they need the same state and constants, if less than eight interpolants are needed then there is room to manually double buffer the interpolants, and allow pipelining of primitives. The real time command stream will still need to manually check that the pipeline has finished with the previous primitive, before writing new data to the parameter memory for the next primitive, while the pipeline works on the current primitive.

For example, the a drawing command in a real-time command buffer might look like this:

```

Wait_for_realtime_pipe_idle           // make sure no real-time command is in the pipeline
Write state reg m in context 7 with data // set rendering state for command
Write state reg m in context 7 with data // set rendering state for command
Write state reg m in context 7 with data // set rendering state for command
Write state reg m in context 7 with data // set rendering state for command
Write const reg at physical address k   // write constant register
Write const reg at physical address k+1 // write constant register
Write const reg at physical address k+2 // etc.
Write vertex 0, parameter 0, in real time parameter store
Write vertex 1, parameter 0, in real time parameter store
Write vertex 2, parameter 0, in real time parameter store
Write setup primitive to primitive assembly (scan converter)
Write initiator register, tag command with 0
Write vertex 0, parameter 8, in real time parameter store
Write vertex 1, parameter 8, in real time parameter store

```

	ORIGINATE DATE 11 March, 2001	EDIT DATE 4 September, 2015	DOCUMENT-REV. NUM. GEN-CXXXXX-REVA	PAGE 13 of 32
--	---	---	--	-----------------------------

Write vertex 2, parameter 8, in real time parameter store
Write setup primitive to primitive assembly (scan converter) // this assumes we double buffer the primitive registers
Write initiator register, tag command with 1
Wait_for_realtime_command_0_not_in_pipe
Write vertex 0, parameter 0, in real time parameter store
Write vertex 1, parameter 0, in real time parameter store
Write vertex 2, parameter 0, in real time parameter store
Write setup primitive to primitive assembly (scan converter)
Write initiator register, tag command with 0
Wait_for_realtime_command_1_not_in_pipe
Write vertex 0, parameter 8, in real time parameter store
Write vertex 1, parameter 8, in real time parameter store
Write vertex 2, parameter 8, in real time parameter store
Write setup primitive to primitive assembly (scan converter) // this assumes we double buffer the primitive registers
Write initiator register, tag command with 1
Wait_for_realtime_command_0_not_in_pipe

5.4 State Management

State management differs from previous ATI chips.

There are eight sets of state registers in the chip. Each pixel or triangle is tagged with which state it is supposed to use. Most of this is hidden from the programmer by the RBBM, which implements the in-order semantics that are normally used. States 0 to 6 are managed by the RBBM for high performance 3D/2D/video rendering. State 7 is reserved for real time commands, and the real time command stream must ensure that the state is not changed while the pipeline is active.

Each register is therefore mapped in to the register space nine times: once for the current state, plus eight additional times to provide access to all existing state. This is only true for the normal pipeline state registers, the constant registers used by the pixel/vertex shaders are handled by a separate, related mechanism.

There are two options for the update of the state registers. The first option is to implement a broadside state copy, which copies the contents of the previous current state to the new current state before the first state write happens to the new state. This is somewhat costly in hardware. The second option is for the state updates done by the driver to be "complete", write the minimum set of state registers that completely defines the new rendering state, this avoids the need for the hw broadside copy.

The constant registers are implemented using a renaming scheme that avoids the need to do a broadside copy when changing state. It also does not use storage for each state, when two state contexts have the same value in the same register, the renaming logic points them at the same physical register.

Since the registers that are most frequently changed are located in the constant memory of the R400 (vertex array pointers, and texture pointers) we may wish to separate updates to the constant registers from general state register updates.

5.5 Bad Data

Bad data can exist for a number of reasons. When a vertex shader does an access to an address which is not permitted (or does not exist) we need a way to avoid hanging, and make debugging possible; A similar issue exists for pixel shaders that do bad texture accesses.

We currently handle a limited form of this: a triangle than contain a vertex which contains (or generates) a NaN or INF is not drawn, it is simply culled at setup.

We will extend this as follows:

For a vertex fetch that goes out of range (or times out) a flag in the vertex is set which will cause that vertex to be treated as if it contained a NaN. A debugging flag will also be set, and if we can find an easy way to do it, the index of the offending vertex will also be stored.

	ORIGINATE DATE 11 March, 2001	EDIT DATE 4 September, 2015	DOCUMENT-REV. NUM. R400 Top Level Spec	PAGE 14 of 32
--	----------------------------------	--------------------------------	---	------------------

For a texture fetch a similar strategy will apply: A bad access will set a flag that will cause the pixel to be dropped. The debugging mode will force the pixel to pass the Z test, and override the color output from the pixelshader with an ugly shade of green.

5.6 Display operation

The display must be able to display from microtiled surfaces and overlays. This will generally force us to adopt line buffers.

The display should support at least two outputs, ideally we will be able to support two high resolution outputs and a low resolution output (TV out)

We will drop support for overlay scaling, and therefore supporting an overlay on all displays becomes affordable, fixing a "bug" that our current dual display products suffer from.

The memory for the line buffers is shared with the L2 texture cache. This allows use a memory size that is closer the maximum requirement of either function, instead of the sum of the maximum requirement.

The maximum resolution color format is 64 bit color for the primary surface and 32 bit color for the overlay
For two 2560 pixel wide line buffers we need

2560 pixels	2560	
two lines	2	
two displays	2	
96 bits of color	96	(32 overlay + 64 bits primary)
total bits	960K bits, 120 Kbytes	

The L2 memory will probably be 128Kbytes, which will leave only 8KB for the texture L2 cache when driving the above display. However, the above case is driving two multi-megapixel displays with the worst case color depth. It works, but 3D performance suffers.

A slightly more normal case might be two 1600x1200 displays, with the same color depth:

2560 pixels	1600	
two lines	2	
two displays	2	
96 bits of color	96	(32 overlay + 64 bits primary)
total bits	600K bits	

which leaves >54Kbytes for the L2 Cache

A benchmark case, one display no overlay, 32 bit color:

2560 pixels	1280	
two lines	2	
two displays	1	
96 bits of color	32	(32 overlay + 64 bits primary)
total bits	81K bits	

which leaves >110Kbytes for the L2 cache.

The line buffers are two scan lines high:

At beginning of an even scan line (scan line 0) 1/2 of line buffer is filled.

The upper half of the buffer is read out, at the same time as the second half of the buffer is filled. When the scanout reaches the midpoint 3/4 of the buffer is filled. When the scanout reaches the end of the scan the buffer is filled. The speed at which the buffer is filled must be greater than 1/2 of the rate at which the buffer is scanned out.

For the odd scanlines, the buffer is completely filled at the start of scanout (as a result of the even scanline finishing properly). As the lower half is scanned out, reads are issued to fetch the data for the next pairs of scanlines. At the end of the odd scan line, the buffer is expected to contain half of the data for the next scan line.

Another way of looking at this is as follows:

	ORIGINATE DATE 11 March, 2001	EDIT DATE 4 September, 2015	DOCUMENT-REV. NUM. GEN-CXXXXX-REVA	PAGE 15 of 22
--	----------------------------------	--------------------------------	---------------------------------------	------------------

At the beginning of the odd scan line the scan buffer is filled. As each word is read from the buffer and sent to the display logic, a request is made to the memory controller to fill in the data. It is not necessary for all the data for the next scan line pair to be fetched by the time that the scan line reaches the end, the real requirement is for the last word in the scan line buffer to get there just before it is read, at the end of the next even scan line.

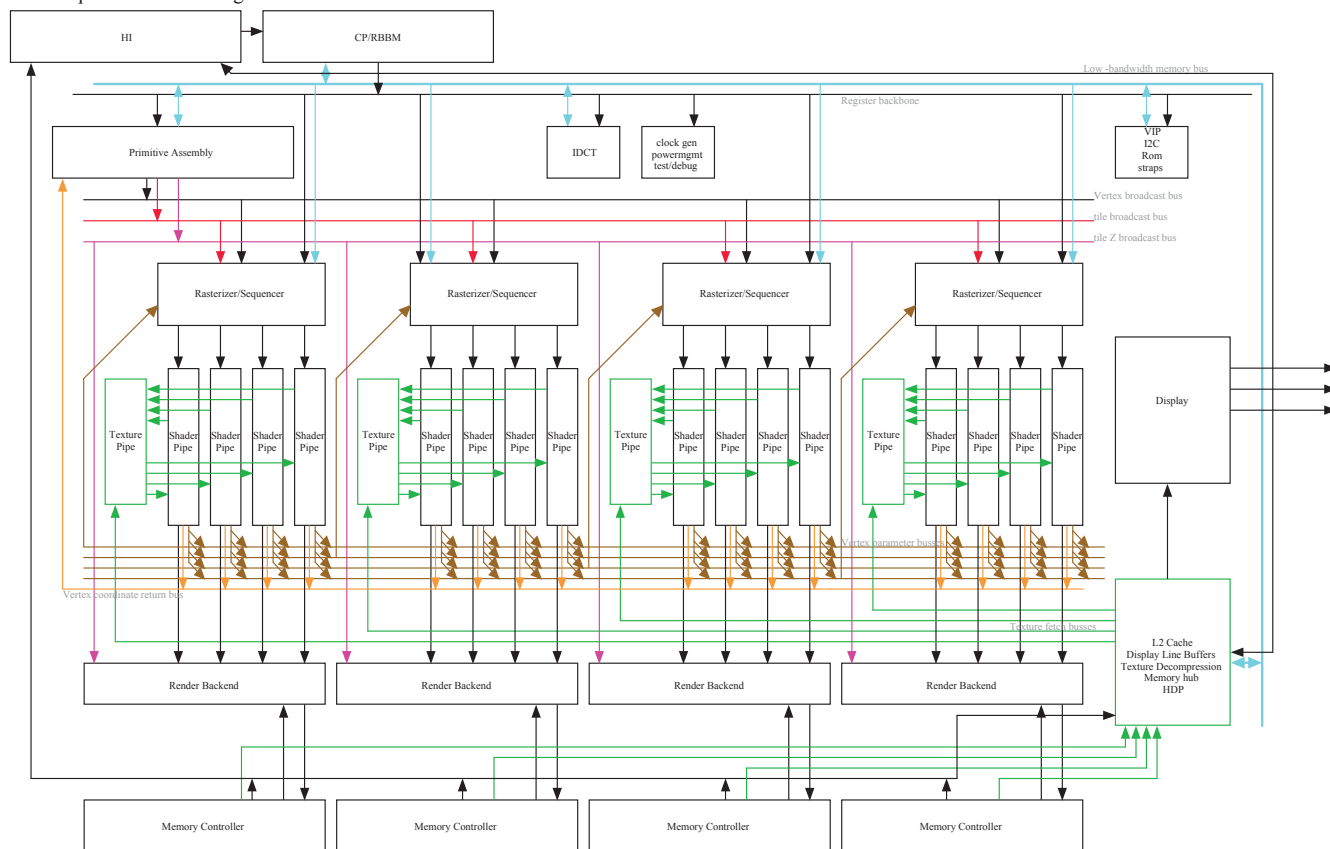
The display lives mostly in the core clock domain. There is a FIFO per pixel clock that crosses into the DAC/TMDS clock domain.

If we are able to implement the time interleaved display block then the display will merge and color convert two pixels per clock in the core clock domain. Whichever display FIFO is closest to empty will get the priority to be filled in the next time slot. The sum of the pixel clocks (display0, display 1, tvout) must be less than 2x the core clock. We should be able to cheat slightly and use some of the horizontal retrace time to fill the display fifo's, this will relax slightly the 2x core clock limit.

Since the 3D pipe is capable of real-time events, such as display triggered scale-blits, we may wish to reconsider the location of several operations that are currently in the display. TV out scaling, and ratiometric scaling for LCD panels may be more cheaply implemented using the 3D pipe instead of dedicated hardware.

6. Block Diagram

R400 Top Level Block Diagram



7. Blocks

HBIU – host bus interface unit
 CP – control processor
 RBBM – register interface manager
 CLK – clock generator
 TC – test controller
 VIP – video input port

	ORIGINATE DATE 11 March, 2001	EDIT DATE 4 September, 2015	DOCUMENT-REV. NUM. R400 Top Level Spec	PAGE 16 of 32
--	----------------------------------	--------------------------------	---	------------------

ROM – boot rom
 I2C – I2C interface
 DU – Display
 MH – Memory Hub
 HDP – Host Data Path
 IDCT – Mpeg decoder
 PA – Primitive Assembly
 TD – Texture Decompression
 RE – Raster Engine
 SP – Shader Pipe
 TP – Texture Pipe
 RB – Render Backend
 MC – Memory Controller

The blocks are combined into a smaller number of blocks for layout:

Layout block	subblocks	Instances R400/450	Instances RV400	Notes
HI	HI	1	1	
CP	CP RBBM CLK Reset	1	1	
Misc	VIP ROM I2C TC	1	1	
DU	DU	1	1	Display
TD	TD MH HDP	1	1	L:2 Cache
PA	PA	1	1	
RE	RE	4	2	
SP	SP	16	8	
TP	TP	4	2	
RB	RB	4	2	
MC	MC	4	2	

8. [Block descriptions](#)

8.1 HBIU – host bus interface unit

The HBIU interfaces the graphics chip to the system AGP bus.

8.1.1 *Description*

The HBIU implements the following buses:

PCI slave
 PCI master
 AGP fast writes
 AGP reads
 AGP writes

64 bit support?

8.1.2 *Major interfaces*

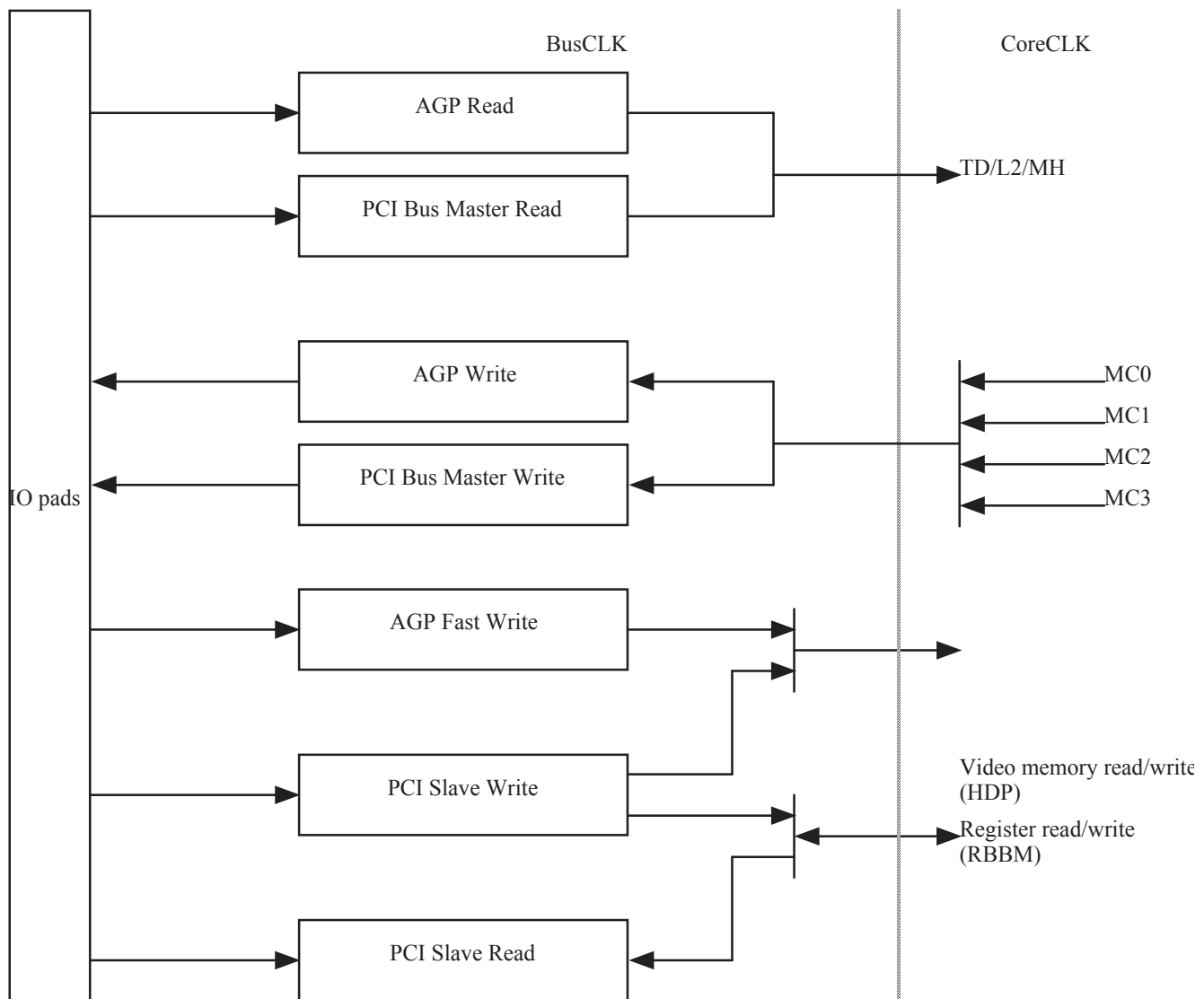
The following busses connect the HBIU to the rest of the chip:

Bus	Chip client	Bus client	Description
-----	-------------	------------	-------------

PROTECTIVE ORDER MATERIAL

	ORIGINATE DATE 11 March, 2001	EDIT DATE 4 September, 2015	DOCUMENT-REV. NUM. GEN-CXXXXX-REVA	PAGE 17 of 22
Host register	CP/RBBM	PCI Slave	CPU reads and writes to chip registers	
Host Data	HDP	PCI Slave AGP fast write	CPU reads and writes to video memory	
AIC Write	MC	PCI Master (writes) AGP writes	Primarily blits to system memory, and control semaphore writes	
AIC Read	TD/MH	PCI Master (reads) AGP reads	CP PM4 reads PA index reads State/vertex program loading Vertex loads AGP texture	
Oddites for VGA				

8.1.3 Block diagram



	ORIGINATE DATE 11 March, 2001	EDIT DATE 4 September, 2015	DOCUMENT-REV. NUM. R400 Top Level Spec	PAGE 18 of 32
--	----------------------------------	--------------------------------	---	------------------

8.2 CP – control processor

8.2.1 Description

The control processor executes the pm4 display list from memory, driving the operation of the rest of the chip. It also implements the real-time event commands.

Currently the CP is based on a custom processor, which has a very limited instruction set and is really only capable of executing the existing program. It is not expected to be capable of doing the translation of 2D packets to the preferred hardware interface, or be able to implement the real time commands.

An alternative is to base the CP on a more generic RISC processor. It appears that this will save area, and make it possible to write the CP control program in C. The ARC core, for example, is less than 20K gates.

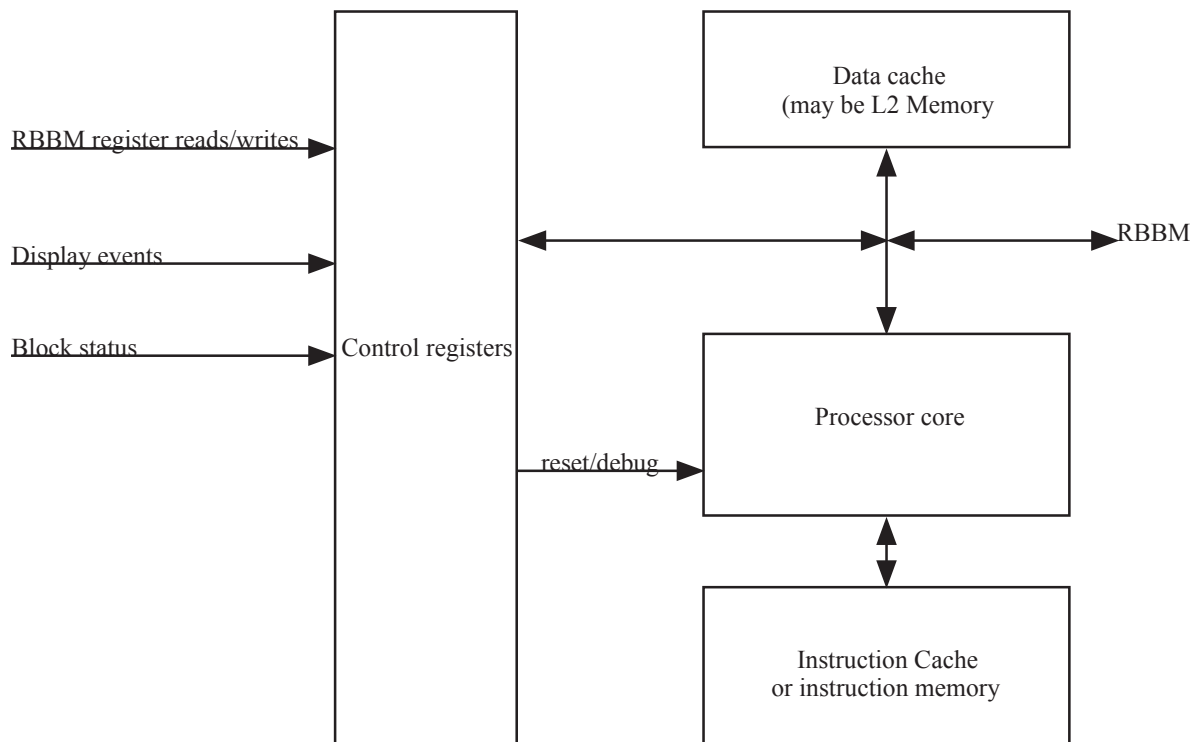
One key change that enables us to consider a processor core instead of the custom PM4 engine is that data is no longer embedded in the command stream. In the R128 to R300 index, vertex, and host-blit data is embedded in the primary ring buffer and the indirect buffer. In the R400 index data is fetched by a dedicated DMA engine in the PA block, and vertex and host blit data is fetched through the texture cache. This allows us to optimized a single path for the data rather than need to optimize both the DMA and PM4 paths. With the CP no longer needing to be able to copy data at 32 bits per instruction (read and write), a less specialized processor can be used.

8.2.2 Major interfaces

Bus	Description
RBBM→CP	Register read write, Used for reset and debugging of CP, and access to control registers
CP→RBBM	Register writes, and reads Register access that occur as a result of executing the control program
CP→MH	Memory reads and writes. Read PM4 buffers, write semaphores to communicate with driver
Display→CP	Source of real time events to trigger real time commands, also delays in command queue based on display status. Current scan line is most common type of data
All block→CP	Blocks status. Used for wait for idle and power down

	ORIGINATE DATE 11 March, 2001	EDIT DATE 4 September, 2015	DOCUMENT-REV. NUM. GEN-CXXXXX-REVA	PAGE 19 of 32
--	----------------------------------	--------------------------------	---------------------------------------	------------------

8.2.3 Block diagram



8.3 RBBM – register interface manager

8.3.1 Description

The RBBM of the R400 is vastly simplified compared to previous versions.

The key differences are:

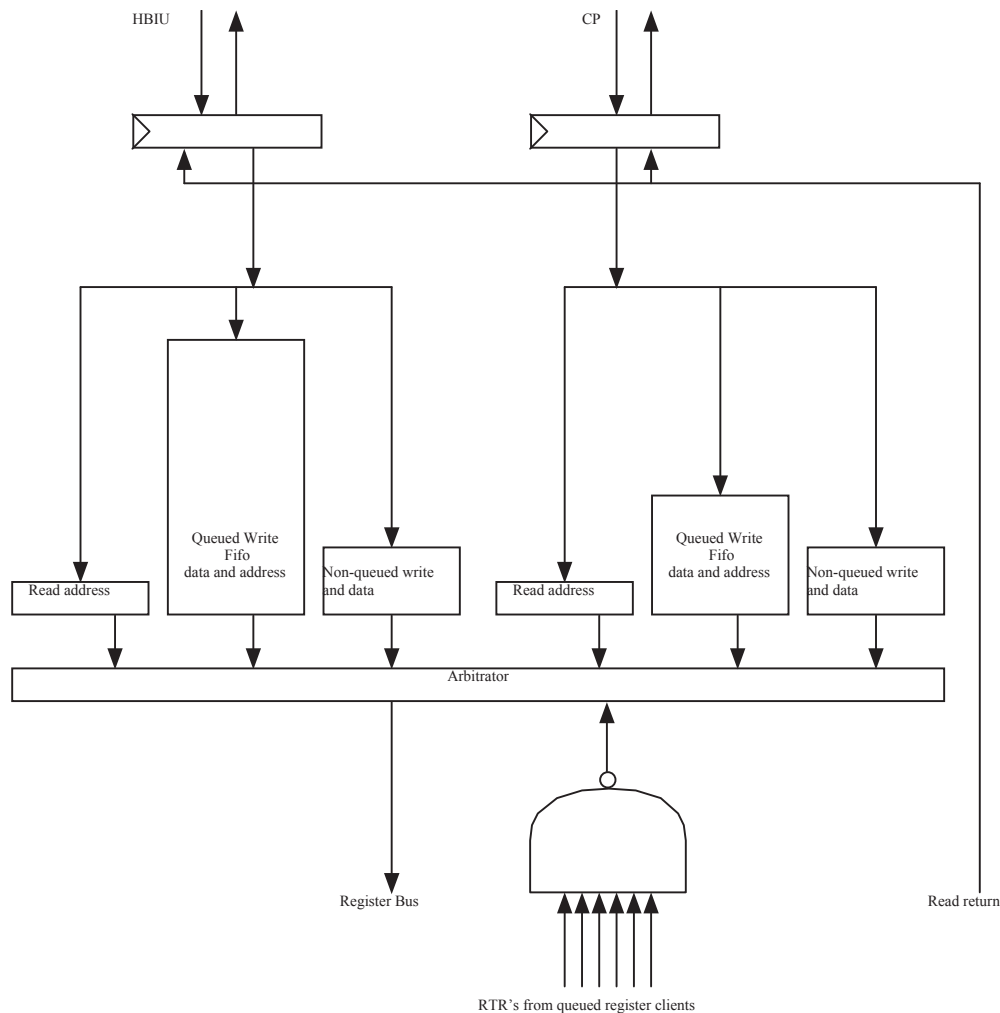
- 1) A much simpler register decoding scheme that does not need the RBBM to be aware of autoreg files.
- 2) A simpler register bus protocol that (for most registers) does not involve any feedback signals to the RBBM
- 3) Support for simple pipelining of the register bus to meet timing goals.
- 4) Much of the synchronization logic that was in the RBBM is now the domain of the CP, this means that bypassing the CP is not a viable production driver mode, but it really is not viable now.
- 5) Power Saving needs some adjustment (since the RBBM is no longer aware of when a block is activated).
- 6) All register bus connections are now single cycle, register to register which will simplify timing.

However the management of state changes has been moved from the blocks in the 3D pipe to the RBBM. The RBBM detects when a state block is no longer in use, tracks the blocks that are not in use, and allocates them to new primitives as needed.

8.3.2 Major interfaces

Bus	Description
HBIU→RBBM	Register read/write
CP→RBBM	Register writes resulting from interpretation of command packets
RBBM-register bus	The purpose of the block
RBBM→CLK	Power management
RBBM→all	Soft/hard reset

8.3.3 Block diagram



8.3.4 RBBM operation

This is copied from the current RBM spec, at some point most of it will be moved back there.

The RBBM has merges register writes and reads from the HBIU and the CP and broadcasts them to the rest of the blocks in the chip. That is all it needs to do.

Registers can either be queued or un-queued. In general queued register writes are initiator registers, or order critical state registers. The RBBM distinguishes between the two types of registers by their address only. The upper 2Kbytes of the register space are queued registers, the remainder is un-queued.

Both the CP and the host can generate both types of register writes.

Un-queued register writes can and will pass queued registers writes. If it is important for un-queued register writes to be held off by a queued register write the host or cp must not send the un-queued register write until the host or cp has determined that the queued register write has completed (usually by a spin lock on a semaphore).

Queued registers are maintained in order from the viewpoint of each originator. I.e. all of the CP's queued writes will complete in order, and all of the hosts will complete in order. There is no ordering between the CP and host- the writes from both clients may become interleaved.

The global register bus is as follows:

Name	Direction	bits	Description
------	-----------	------	-------------

PROTECTIVE ORDER MATERIAL

	ORIGINATE DATE 11 March, 2001	EDIT DATE 4 September, 2015	DOCUMENT-REV. NUM. GEN-CXXXXX-REVA	PAGE 21 of 22
--	----------------------------------	--------------------------------	---------------------------------------	------------------

WE	RBBM→	1	Write enable, address and data are valid
Addr[19:2]	RBBM→	18	Register address
Wm[3:0]	RBBM→	4	Register write mask, should be ignored by most clients
Wd[31:0]	RBBM→	32	Data
RE	RBBM→	1	Read Enable, address is valid
Rd[31:0]	→RBBM	32	Read data returned
RRn	→RBBM	1	Read return strobe (active low)

The protocol for a write is simple:

On a rising edge, if WE is high then the data and address is valid.

There are no completion signals, there is no way to abort a write.

Handshake signals for queued registers will be described later and are separate from the register bus.

The read protocol is somewhat more complex.

A read request is sent out when RE is high. The address holds the address of the read request.

RE and addr will only be valid for one clock cycle.

Some number of clock cycles later the RBBM will receive the return data back, when RR is low.

The read return "bus" (Rd and RRn) is the logical AND of all the clients that can respond to a read request.

All clients but the client that is responding to the read request drive a logical 1 on the bus. The wiring of the read return bus is a tree of point to point connections, and each node one or more sub-busses are AND'd together, registered, and driven on to the RBBM. This is the same as an OR tree, but the signal is inverted. Since a read return cycle is surrounded by idle cycles the only critical transition is high to low for the Rd signals (possible timing help, at the cost of needing to tell static timing to ignore low to high transitions).

Only one read is outstanding at any time. Reads will pass queued writes, (should/can they pass all writes?)

The RBBM will timeout on a read after 64? clocks. It is critical that no client respond latter than 64 clock as the RBBM may timeout on the read, issue another and interpret the very late response of the first read as the second read.

If a read times out the RBBM will return dummy data (such as '0xDEADBEEF') to the requestor and mark in a debug register that an error happened.

Both queued and non-queued register writes are broadcast on the same bus. To implement the queued registers the RBBM looks at the status of all of the RTR signals from the clients that contain queued registers. Only if all are high will any queued register be allowed to issue from the RBBM. Note that these signals are registered on the boundary of the RBBM, and the register bus is also registered. This means that there is at least a two clock latency responding to a RTR signal deasserting. Since the clients will also be registered this means that a client will receive four or more queued register writes after asking them to stop. It is the clients responsibility to have enough buffering so that no register writes are lost.

8.4 CLK – clock generator

8.4.1 Description

The clock generator block generates the many clocks used by the R400:

Clock	speed range	
AGP	133 to 533	AGP clock
Sclk	33to 500 Mhz	core clock
Mclk	33 to 500	memory clock
P0clk	10 to 450	pixel clock for primary display (5x faster for TMDS/LVDS)
P1clk	10 to 450	pixel clock for secondary display (5x faster for TMDS/LVDS)
Tvclk		pixel clock for tvout

8.4.2 Major interfaces

Bus	Description
-----	-------------

	ORIGINATE DATE 11 March, 2001	EDIT DATE 4 September, 2015	DOCUMENT-REV. NUM. R400 Top Level Spec	PAGE 22 of 32
RBBM→CLK	control			

8.4.3 *Block diagram*

8.5 TC – test controller

8.5.1 *Description*

8.5.2 *Major interfaces*

8.5.3 *Block diagram*

8.6 VIP – Video input port

8.6.1 *Description*

8.6.2 *Major interfaces*

Bus	Description
VIP→MH	DMA transfers
RBBM→VIP	Control

8.6.3 *Block diagram*

8.7 ROM – boot rom

On powerup the graphics chip reads the straps from the rom. The rom is then used for responding to boot rom read requests from the PCI bus.

We will only support serial roms.

The list of supported roms is TBD.

8.7.1 *Description*

8.7.2 *Major interfaces*

Bus	Description
ROM→HBIU	Boot rom read interface
RBBM→ROM	Flash/eeprom boot rom write interface
ROM→chip	Decoded straps

8.7.3 *Block diagram*

8.8 I2C – I2C interface

8.8.1 *Description*

The I2C bus is a 2 wire bus used to communicate with other multimedia devices (such as tv tuners)

8.8.2 *Major interfaces*

Bus	Description
RBBM→I2C	Read/write interface

	ORIGINATE DATE 11 March, 2001	EDIT DATE 4 September, 2015	DOCUMENT-REV. NUM. GEN-CXXXXXX-REVA	PAGE 23 of 32
--	----------------------------------	--------------------------------	--	------------------

8.8.3 Block diagram

8.9 DU – Display

The R400 display drives up to three displays: two monitors and a TVOUT.

The chip can have as much as two analog RGB DAC's, two dual channel TMDS outputs, and one dual channel LVDS output.

All support for the scaling overlay is removed. The display supports a non-scaling overlay on each display.

See display operation section above for more details.

8.9.1 Description

Hopefully we have the resources to move to the time interleaved display design.

The following frame buffer formats are supported:

Primary surface:

8bpp index

16bpp 4444,565.555 RGB

32bpp 8888 RGB

64bpp 16:16:16:16, either sRGB or the R400 floating point format

Overlay:

32bpp 8888 RGB

4:2:2 YUYV

the color conversion for the overlay is controlled by a programmable matrix, so the choice of color space is arbitrary.

The maximum display pixel clock is greater than 400 MHz.

If we build the time interleaved design, then the maximum number of display pixels will be 2x the core clock speed.

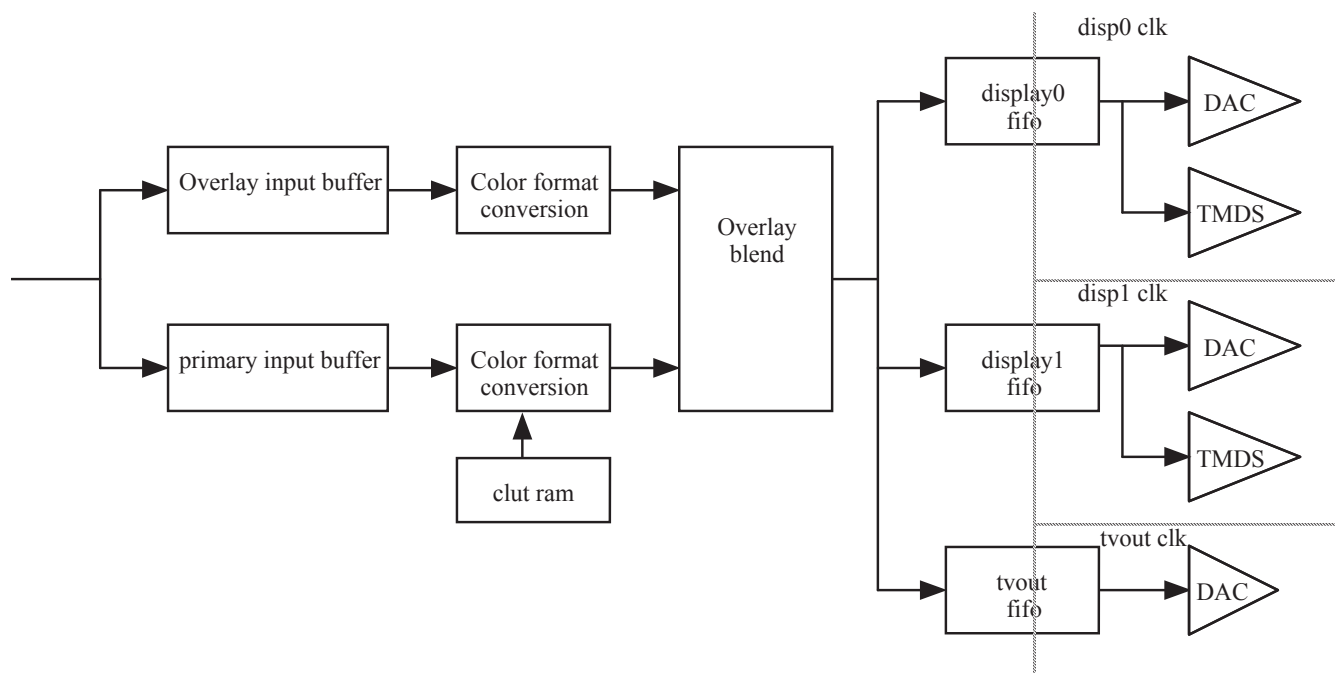
This will be divided among the two displays and tvout.

8.9.2 Major interfaces

Bus	Description
TD→DU	Memory read interface
RBBM→DU	Register writes/reads
DU→CP	Synchronization information

	ORIGINATE DATE 11 March, 2001	EDIT DATE 4 September, 2015	DOCUMENT-REV. NUM. R400 Top Level Spec	PAGE 24 of 32
--	----------------------------------	--------------------------------	---	------------------

8.9.3 Block diagram



8.10 MH – Memory Hub

8.10.1 Description

The memory hub acts as a switch between the many small clients of the memory controllers, and the two or four memory controllers. This allows most blocks to not have any dependencies on the number of memory controllers.

8.10.2 Major interfaces

The memory hub has a 32 bit read and a 32 bit write bus to each of the memory controllers. If we co-locate the MH and the L2 cache in the texture decompression block, then the 128 bit read return bus from the MC to the L2 cache can be used to return read data to the MH instead of a private bus.

The clients of the MH:

HDP
CP
VIP
PA- index buffer reading
RE- Vertex/pixel program loads, hierarchical Z
IDCT
?

8.10.3 Block diagram

8.11 HDP – Host Data Path

The host data path allow the host cpu to access video memory. It provides eight “surfaces” that provide endian and tiling translation, making the target area of memory look like a linear surface in the processors native endian.

The HDP also implements most of the legacy VGA functionality.

	ORIGINATE DATE 11 March, 2001	EDIT DATE 4 September, 2015	DOCUMENT-REV. NUM. GEN-CXXXXX-REVA	PAGE 25 of 32
--	----------------------------------	--------------------------------	---------------------------------------	------------------

8.11.1 *Description*

8.11.2 *Major interfaces*

Bus	Description
HI→HDP	Memory read/write requests to HDP
HDP→MH	HDP reads/writes to local memory

8.11.3 *Block diagram*

8.12 IDCT – Mpeg decoder

8.12.1 *Description*

The R400 uses the same implementation of IDCT/MPEG as the R300. This block decodes the compressed stream, placing the resulting IDCT data in one buffer in memory, and the motion vectors in another. The 3D pipe is then programmed to read the motion vectors and IDCT data and complete the decoding operation

8.12.2 *Major interfaces*

Bus	Description
IDCT→MH	Read command stream, write IDCT results and motion vectors

8.12.3 *Block diagram*

8.13 PA – Primitive Assembly

8.13.1 *Description*

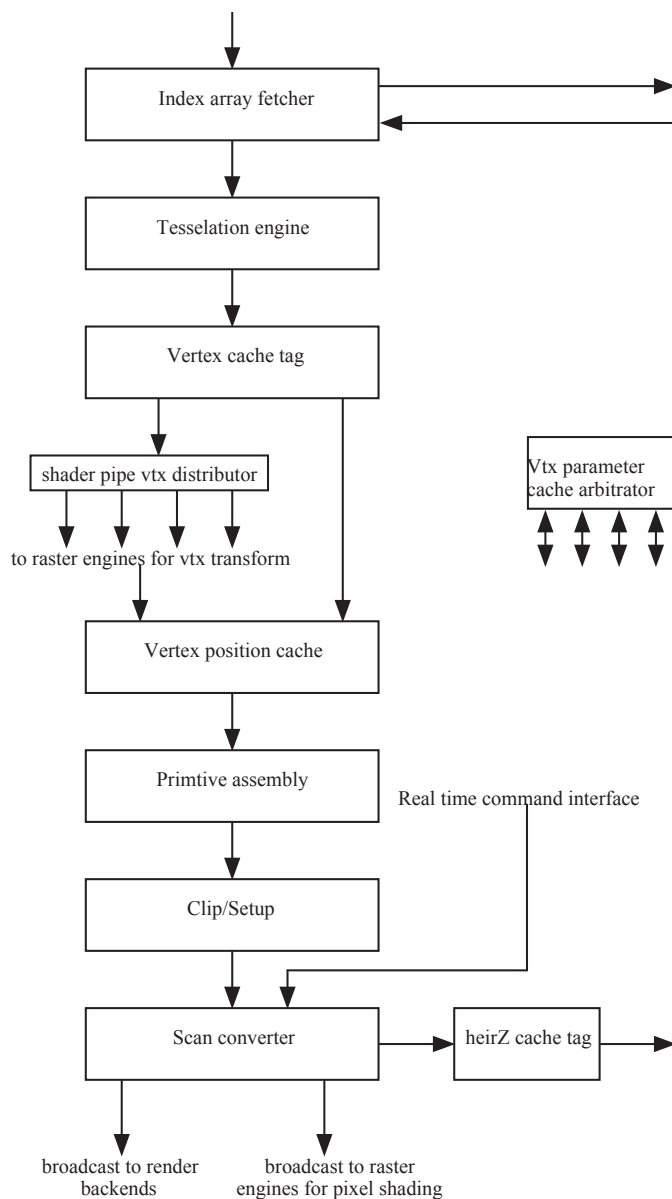
The primitive assembly block fetches or creates the indices to vertices, possibly creating extra vertices with the tessellation engine. It determines which vertices have not been recently seen (and will therefore not be located in the post transform vertex cache), assembles vectors of sixteen vertices that need to be transformed, and submits them to a raster engine/shader pipe set to be transformed. It then receives the transformed vertex position data from the shader pipes. The vertex cache tag also outputs the sequence of cache addresses generated from the incoming indices. The primitive assembly subblock then creates primitives (lines, points, rectangles, triangles) from the vertices. It also implements the line counter for styled lines. The primitives are setup in the setup/clip block, but first clipped to the view frustum and optionally the user clip planes. The scan converted does a coarse walk of the primitive using an 8x8 grid. The scan converted also determines when the hierarchical Z data needed for culling will not be in the local hierarchical z cache in each rasterizer and makes the needed memory read requests. An arbitrator, centrally located in the PA block arbitrates each rasterizers reads from the post transform vertex parameter cache, which is distributed among the shader pipes.

8.13.2 *Major interfaces*

Bus	Description
RBBM→PA	State changes, and initiator register writes
PA→MH	Index fetch path
PA→RE	Vertex transform packets
PA→MH	Hierarchical Z read request
PA→RBn	Coverage mask, position, and Z slope
PA→SPn	8x8 tiles to be rasterized

	ORIGINATE DATE 11 March, 2001	EDIT DATE 4 September, 2015	DOCUMENT-REV. NUM. R400 Top Level Spec	PAGE 26 of 32
--	----------------------------------	--------------------------------	---	------------------

8.13.3 Block diagram



8.14 TD – Texture Decompression

8.14.1 Description

The texture decompression block converts the memory texture formats into the uncompressed texture formats supported by the texture pipes. It consists of the L2 texture cache, the texture decompression logic, a set of output buffers, and the texture addressing logic.

The decompressed formats supported are:

- 32 bpp (8888) unsigned
- 32 bpp (8888) signed
- 32 bpp (16,16) unsigned and signed
- 32 bpp (32) unsigned and signed

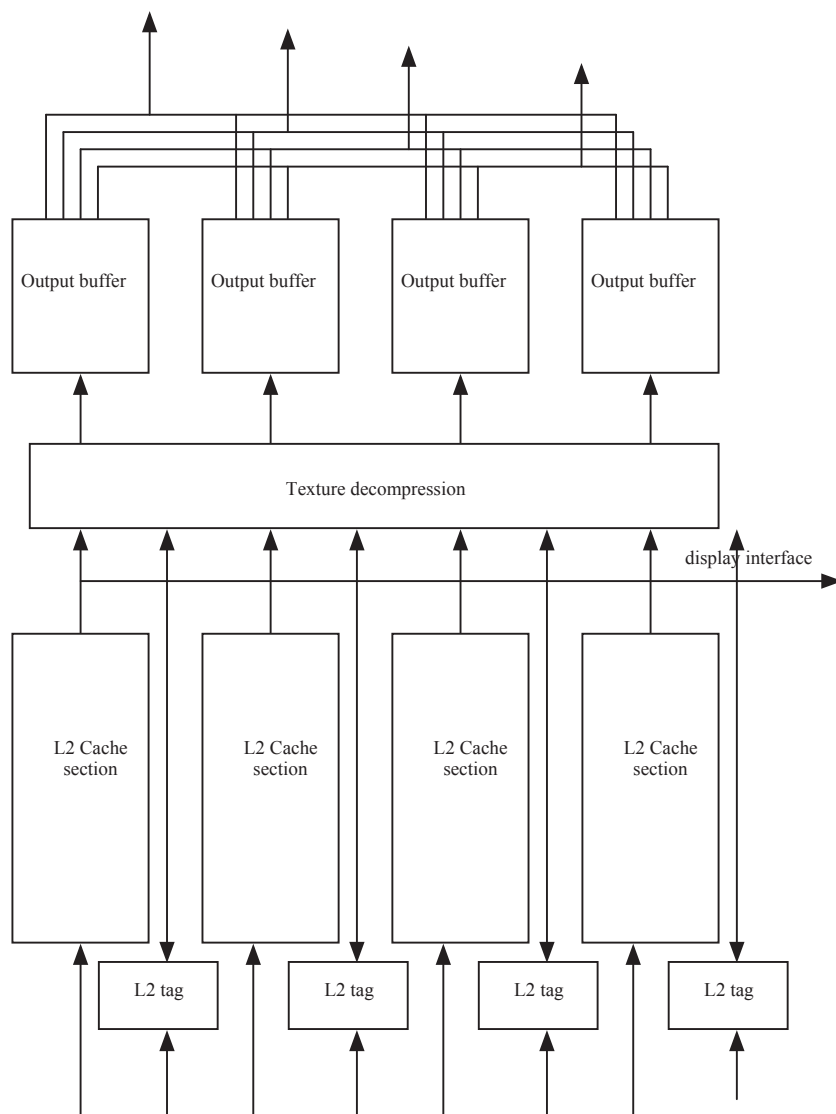
we may also support a 8bpp mono format to improve the performance of shadow buffering.

	ORIGINATE DATE 11 March, 2001	EDIT DATE 4 September, 2015	DOCUMENT-REV. NUM. GEN-CXXXXX-REVA	PAGE 27 of 32
--	----------------------------------	--------------------------------	---------------------------------------	------------------

8.14.2 Major interfaces

Bus	Description
TP→TD	Texture requests and returned data
TD→MCn	Memory read requests
TD→Display	Data path for display which uses the L2 cache as its line buffers
MCn→TD	Invalidate snoop bus for cache coherency

8.14.3 Block diagram



8.15 RE – Raster Engine

8.15.1 Description

The raster engine performs two duties: it does the detail walk of 8x8 tiles of primitives, and it contains the sequencer for the shader pipe.

The shader pipe has the FIFO to allow for balance between the pipelines in the chip, it appears that this FIFO is 64 8x8 tiles deep. Only tiles that are owned by this pipeline are stored in the FIFO, others are immediately rejected. When an 8x8 tile is read out of the FIFO, it is checked against the heir-Z fail data that has arrived in the local heir-Z cache. If the primitive fails, it is rejected and the RB is informed that the tile has been killed.

	ORIGINATE DATE 11 March, 2001	EDIT DATE 4 September, 2015	DOCUMENT-REV. NUM. R400 Top Level Spec	PAGE 28 of 32
--	----------------------------------	--------------------------------	---	------------------

We are going to support hierarchical Z object culling within the command stream. To support this we have the ability to draw a bounding object, heir-Z test it, but kill it before we rasterize it. The raster engine will receive tiles that are marked indicating that they are part of an occlusion query, and test them against the heir-z memory. All the tiles are rejected, but if any of them pass the heir-Z test then id then a flag is set. When the marker (which is an id) changes, indicating the end of this occlusion query, the RE will signal back to the primitive assembly if the flag was set or not.

If the tile passes the heir-z test we need to ensure that the parameter data needed to interpolate the triangle is either in the local IO parameter cache, or on its way there. If not a request needs to be made to the arbitrator in the PA to get the needed data/

A FIFO on the output of the HZ cull and parameter cache tag buffers the passing tiles while waiting for the parameter data to arrive at the cache. It also provides buffering so that the rest of the pipeline can stay busy during a long string of tiles that fail the hierarchical Z test.

The next step is a detail walker that generates the coverage mask for each potentially covered 2x2 quad in the 8x8. We may need a path from this result to the render backend to aid in its determination as to what to fetch. The parametric coordinates are calculated, and used to driver the interpolator. We need to be able to do both perspectivly correction interpolation and non-corrected interpolation.

The raster engine breaks the stream of pixels into 4 quad vectors (16 pixels) and will wait until the needed space is available in the shader pipe, and then start the sequencer running the pixel shader program.

The second part of the raster engine is the sequencer.

The sequencer first arbitrates between vectors of 16 vertices that arrive directly from primitive assembly and vectors of 4 quads (16 pixels) that are generated in the raster engine.

The vertex or pixel program specifies how many GPR's it needs to execute. The sequencer will not start the next vector until the needed space is available.

The sequencer is based on the R300 design. It chooses an ALU clause and a texture clause to execute, and execute all of the instructions in a clause before looking for a new clause of the same type. Each vector will have eight texture and eight alu clauses, but clauses do not need to contain instructions. A vector of pixels or vertices ping-pongs along the sequencer FIFO, bouncing from texture reservation station to alu reservation station. A FIFO exists between each reservation stage, holding up vectors until the vector currently occupying a reservation station has left.. A vector at a reservation station can be chosen to execute. The sequencer looks at all eight alu reservation stations to chose a alu clause to execute and all eight texture stations to chose a texture clause to execute. The arbitrator will give priority to clauses/reservation stations closer to the top of the pipeline. It will not execute an alu clause until the texture fetches initiated by the previous texture clause have completed.

To support the shader pipe the raster engine also contains the shader instruction cache and constant store.

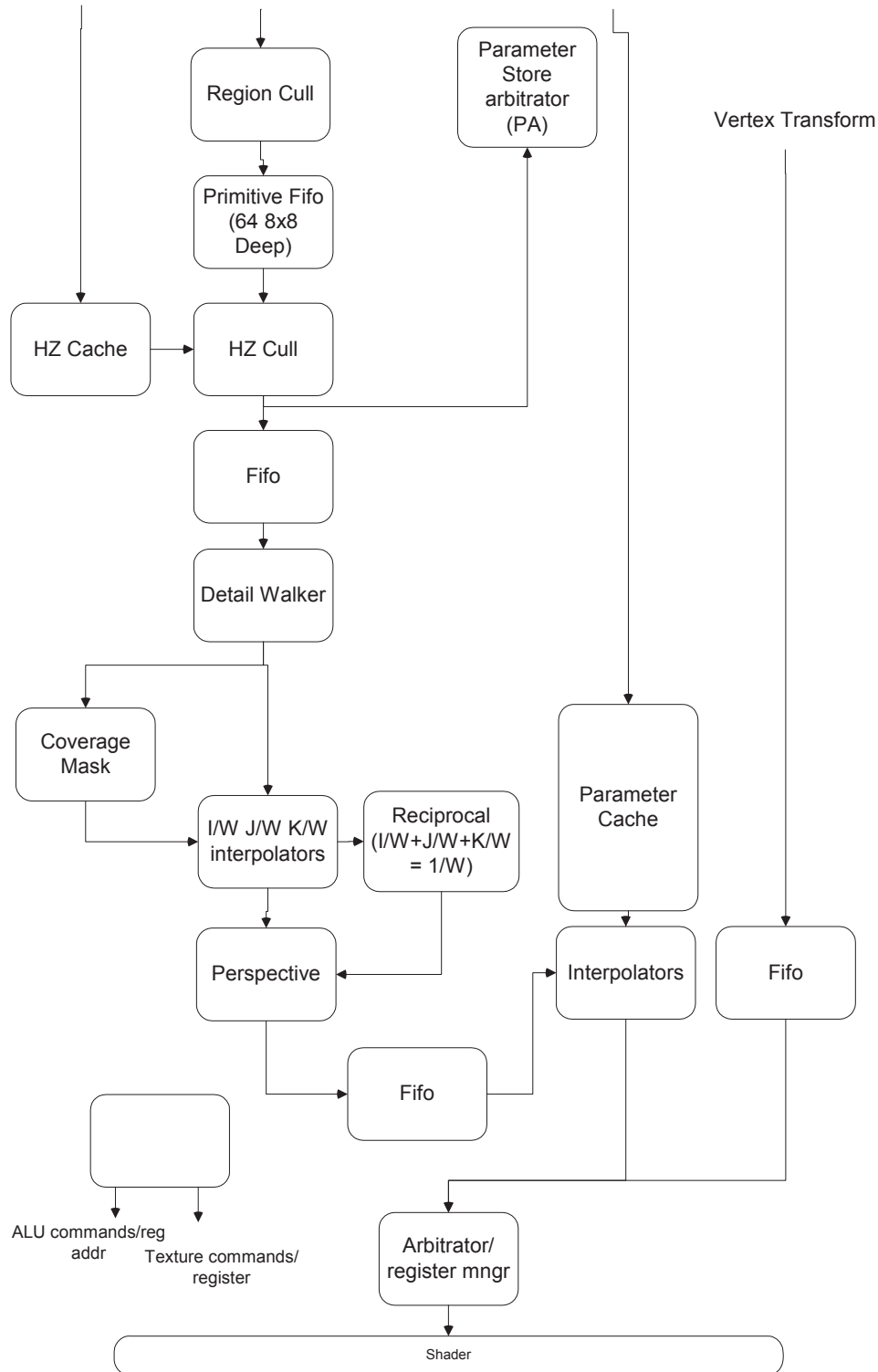
8.15.2 Major interfaces

Bus	Description
PA(sc)→RE	Broadcast bus for 8x8 slices of primitives. I,J,K plane equations, front most Z for heir-Z culling, pointer for location of parameter data in vertex parameter cache
MH→RE	Returned hierarchical Z data for local cache.
PA→RE	Parameter request port
SC→RE	Returned parameter data
PA→RE	Requests to transform packets of vertices
RBBM→RE	State register reads/writes
RE→SC	Interpolated parameter data
RE→SC	Instructions, constants, register file addresses
RE→RB	Heir-Z pass/fail information
RE→RB	Sequencing information for availability of pixels
RE→PA	Sequencing interface for returning transformed vertices.
RE→PA	Occlusion query results
MH→RE	Shader I Cache fills

	ORIGINATE DATE 11 March, 2001	EDIT DATE 4 September, 2015	DOCUMENT-REV. NUM. GEN-CXXXXXX-REVA	PAGE 29 of 32
--	--	--	--	--------------------------

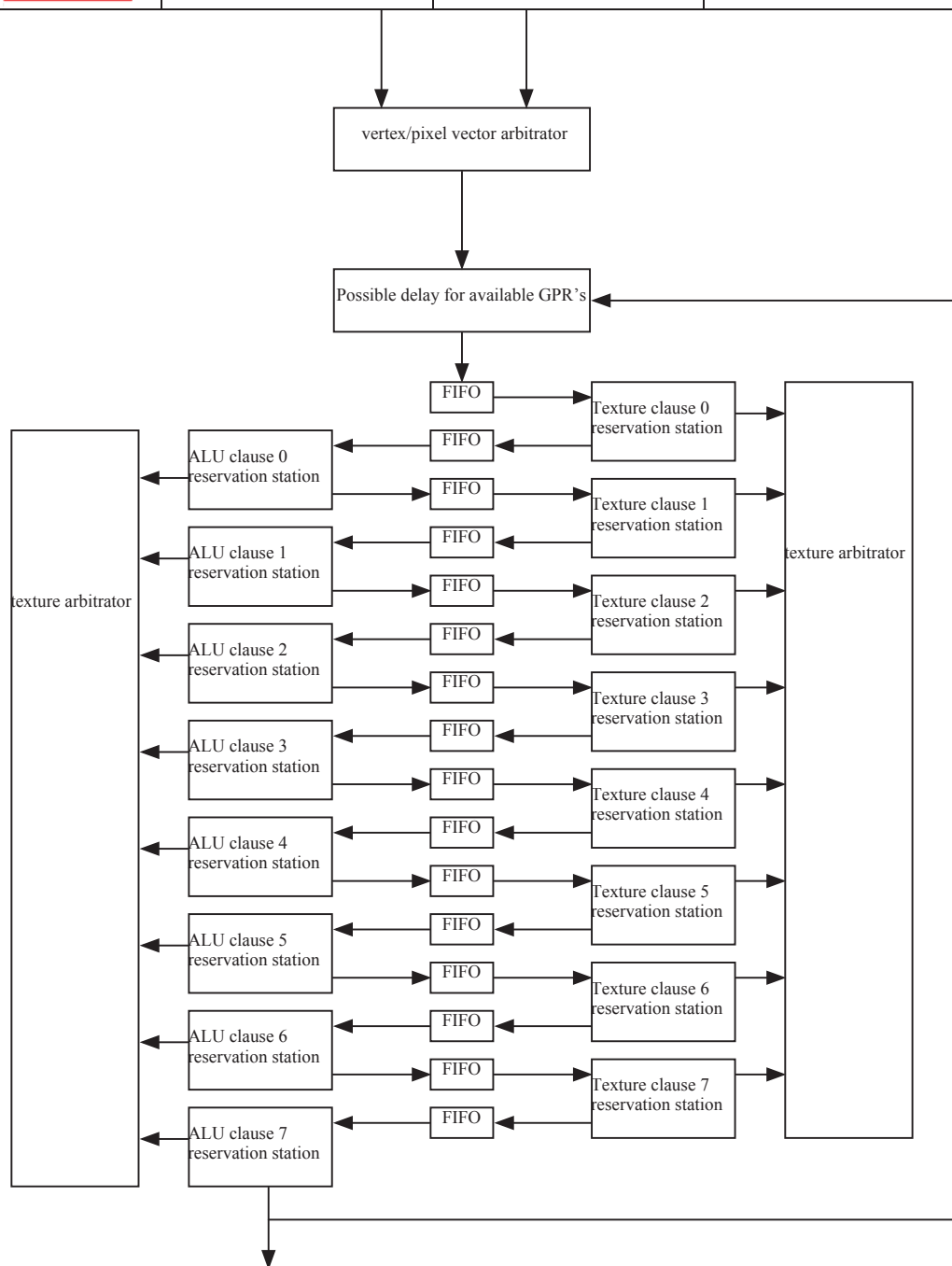
8.15.3 Block diagram

8.15.3.1 RE Block diagram



8.15.3.2 RE sequencer

8.15.3.3 RE sequencer arbitrator



8.16.1 Description

The shader pipe contains four floating point MAC's, and an

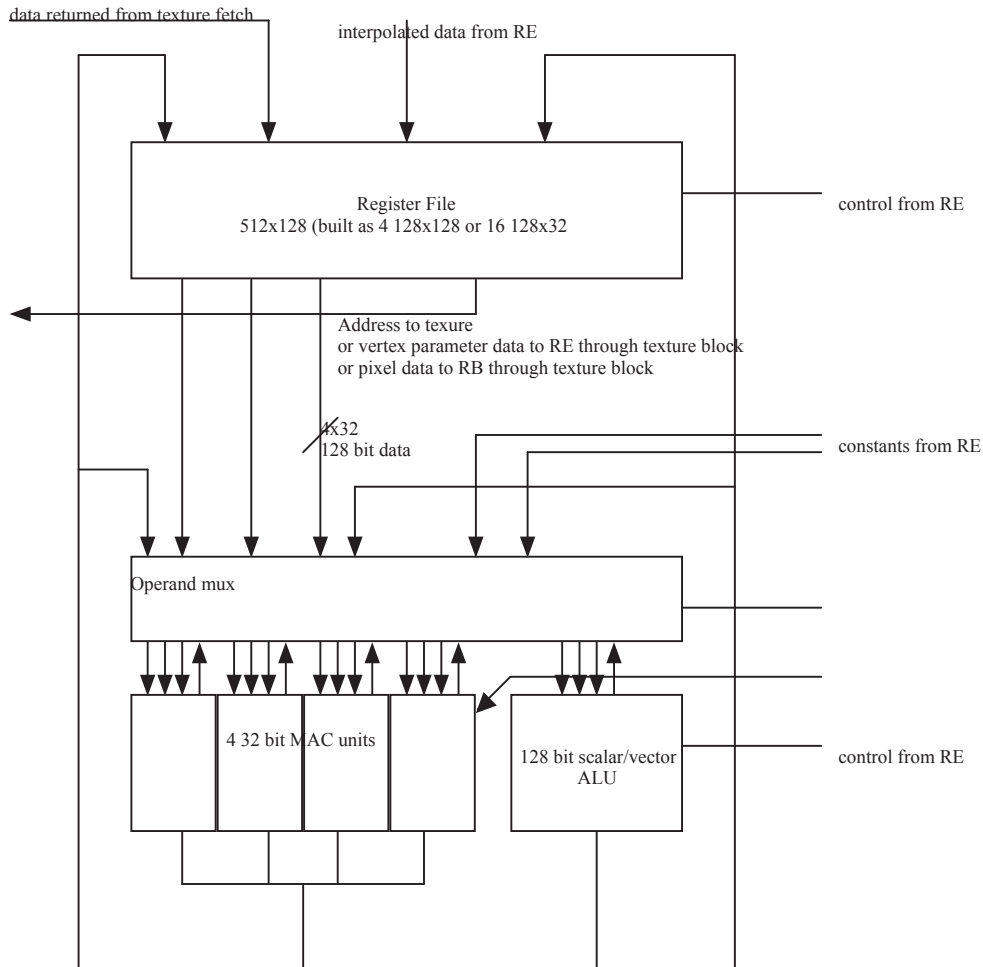
8.16.2 Major interfaces

Bus	Description
RE→SP	Interpolated data
RE→SP	Control

PROTECTIVE ORDER MATERIAL

	ORIGINATE DATE 11 March, 2001	EDIT DATE 4 September, 2015	DOCUMENT-REV. NUM. GEN-CXXXXXX-REVA	PAGE 31 of 32
SP→TX	Texture requests + vertex parameters + pixels to render backend			
TX→SP	Returned texture data			
RE→SP	Constants			
SP→SP	Local w bus for derivative opcode			

8.16.3 Block diagram



	ORIGINATE DATE 11 March, 2001	EDIT DATE 4 September, 2015	DOCUMENT-REV. NUM. R400 Top Level Spec	PAGE 32 of 32
--	----------------------------------	--------------------------------	---	------------------

8.17 TP – Texture Pipe

8.17.1 *Description*

8.17.2 *Major interfaces*

8.17.3 *Block diagram*

8.18 RB – Render Backend

8.18.1 *Description*

8.18.2 *Major interfaces*

8.18.3 *Block diagram*

8.19 MC – Memory Controller

8.19.1 *Description*

8.19.2 *Major interfaces*

8.19.3 *Block diagram*

9. Common Foundations

9.1 Logic Design

9.1.1 *Data formats*

As much as possible, data should be stored and processed identically to x86 (or sparc) conventions. This will, for example, allow the emulator to use normal 32 bit floats and the processors native multiply, add and other operations. This will have a significant effect on the achieved simulation performance compared to being “almost” identical which requires the emulator take several operations to match the hardware bit-exact.

9.1.2 *Register Bus*

Issues:

32 vs. 64 bit

Do rendering state updates happen on this bus or over a dedicated path from memory?

Flow control

9.1.3 *Block Communication protocol*

We want to specify a limited number (one is probably not possible) number of different ways that blocks are interconnected to simplify verification and emulation.

9.2 Software